Projet REOP 2025/2026 - Groupe 21

Siméon DEMOURY - Noé HAGELAUER - Amine EL ALAMI

1 Questions

Question 1 - 3-index formulation (MILP) :
On pose $K = \{1, ..., K_m\}$ les véhicules disponibles, et M une constante suffisamment grande. On pourra considérer par exemple $M = \max_{i \in I} t_i^{max} + \max_{i,j \in I} \tau(i,j) + \max_{i \in I} l_i$.
On considère la variable x_{ij}^k comme définie dans l'énoncé (en fixant $x_{ii}^k = 0$ pour tout $i \in V, k \in K$), et t_i^k l'instant où le véhicule $k \in K$ arrive à $i \in I$.
On peut modéliser le problème comme suit : $\min_{x_{ij}^k} \sum_{k \in K} \sum_{i,j \in V} \delta^M(i,j) x_{ij}^k$, avec :
- conservation du flot : $\forall j \in V, \forall k \in K, \quad \sum_{i \in V} x_{ji}^k = \sum_{i \in V} x_{ij}^k$;
- contrainte capacitaire : $\forall k \in K, \quad \sum_{i \in I} w_i (\sum_{j \in V} x_{ji}^k) \leq Q$;
- contraintes temporelles : $\forall i, j \in V, \forall k \in K, \quad t_j^k \geq t_i^k + l_i + \tau(i,j) - M(1 - x_{ij}^k)$,
$t_i^{min} \leq t_i^k \leq t_i^{max}$ et $t_0^k = 0$;
- unicité de la visite : $\forall i \in I, \quad \sum_{k \in K} \sum_{j \in V} x_{ji}^k = 1$.

Question 2 - 2-index formulation (MILP) :
On considère la variable x_{ij} comme définie dans l'énoncé (en fixant $x_{ii} = 0$ pour tout $i \in V$), et $u_i \geq 0$ la charge cumulée transportée pour chaque client i.
On peut formuler le problème comme suit : $\min_{x_{ij}} \sum_{i,j \in V} x_{ij} \delta^M(i,j)$, avec :
- conservation du flot et visite unique : $\forall j \in I, \sum_{i \in I} x_{ij} = \sum_{i \in I} x_{ji} = 1$;
- contraintes MTZ : $\forall i, j \in I, u_j - u_i \geq w_j - Q(1 - x_{ij})$ et $w_i \leq u_i \leq Q$.

Question 3b - Dynamic Programming (Single Route) :
- États possibles : $\bigcup_{k \in [|S|]} \{(s_{i_1}, ..., s_{i_k}), i_1, ..., i_k \in [|S|]\}$.
- Actions possibles pour un état $(s_{i_1}, ..., s_{i_k})$:
$A((s_{i_1}, ..., s_{i_k})) = \{s_j \in S, j \neq i_1, .., i_k, \quad \forall h < m \in [k], \sum_{w=h}^{m-1} \tau_f(i_w, i_{w+1}) + l_w + L_m \leq t_h^{max} - t_h^{min}\}$.
Pour $k = |S|, \quad A((s_{i_1}, ..., s_{i_k})) = S_0$, avec $l_0 = 0, t_0^{min} = 0, t_0^{max} = +\infty$.

- Fonction coût pour un état $(s_{i_1}, ..., s_{i_k})$: $c((s_{i_1}, ..., s_{i_k}), s_j) = c^{fuel} \delta^M(i_k, j)$.

- Fonction de valeur $V((s_{i_1}, ..., s_{i_k}))$: coût d'une solution partielle où les commandes $(s_{i_1}, ..., s_{i_k})$ ont été réalisées dans cet ordre, avec $V(\emptyset) = 0$.

- Relation de récurrence : $V((s_{i_1}, ..., s_{i_k})) = \min_{s' \in A((s_{i_1}, ..., s_{i_k}))} \{c((s_{i_1}, ..., s_{i_k}), s') + V((s_{i_1}, ..., s_{i_k}, s')\}$.

Question 4 - Lower bound :
On peut considérer la borne inférieure "brutale" suivante : $\lceil \frac{\sum_i w_i}{Q} \rceil$. Des solutions moins brutales seraient de considérer les solutions SOL_{NF} et SOL_{FF} des algorithmes NEXT-FIT et FIRST-FIT, telles que $\frac{SOL_{NF}+1}{2}$ et $\frac{2SOL_{FF}}{3}$ sont des bornes inférieures. On se rapporte alors à du *bin packing*.

2 Heuristique Constructive

Pour générer une solution de base efficacement, on utilise une **heuristique gloutonne**.

1. **Tri :** Les commandes sont triées par ordre croissant de fenêtre de temps pour minimiser les temps d'attente et faciliter le respect des contraintes horaires. *Comp:* $\mathcal{O}(N \log N)$
2. **Insertion :** On parcourt la liste triée et on ajoute chaque commande à la tournée courante. On sélectionne la famille de véhicule parmi un ensemble restreint de candidats (les moins chers) qui satisfait les contraintes de capacité et de temps. Si aucun véhicule ne convient, la tournée est fermée et une nouvelle est ouverte. Le coût de vérification dépend de la longueur de la tournée m. *Comp:* $\mathcal{O}(N \cdot m)$

3 Opérateurs Heuristiques et Complexité

Le solveur se base sur une architecture de **Large Neighborhood Search (LNS)**. On pose N les commandes, K les routes, et $m \approx N/K$ la taille moyenne d'une route.
Définitions : Pour éviter une complexité trop élevée, on utilise des constantes pour borner l'exploration des voisinages : p limite le nombre de positions d'insertion testées dans une route, k restreint le choix aux familles de véhicules les plus prometteuses, et r borne le nombre de routes cibles évaluées pour chaque mouvement. De plus, s définit la taille maximale des tournées candidates à l'élimination complète et q représente le nombre de clients retirés lors de la phase de Ruine.
Note sur la Complexité Globale : L'algorithme est piloté par le temps (arrêt à $T_{\max}$) donc la complexité globale n'est pas définie. La performance dépend du nombre d'itérations réalisées dans le temps imparti, lui-même dépendant du coût des opérateurs ci-dessous.

3.1 Fonctions d'Évaluation

— **Feasibility Check :** Simule les tournées chronologiquement en mémorisant la capacité accumulée et les temps d'arrivée. Retourne la marge avant violation pour tester la robustesse de la tournée. *Comp:* $\mathcal{O}(m)$
— **Route Cost :** Calcule les coûts de location, carburant et rayon. Le calcul du diamètre (distance max entre deux points quelconques) domine le coût d'évaluation. *Comp:* $\mathcal{O}(m^2)$

3.2 Fonctions de Destruction (Ruin)

Utilisées pour échapper aux optima locaux en retirant des clients vers un "pool" d'orphelins.
— **RuinRandom :** Supprime une route aléatoirement (pour forcer une réduction de la flotte) et q clients aléatoires pour créer des opportunités d'insertion. *Comp:* $\mathcal{O}(N)$
— **RuinWorstLight :** Calcule un score pour chaque client basé sur sa contribution à la distance et la largeur de sa fenêtre de temps. Trie et retire les q clients ayant les scores les plus élevés (les "maillons faibles"). *Comp:* $\mathcal{O}(N)$ score $+ \mathcal{O}(N \log N)$ trie

3.3 Réparation & Construction

— **Repair (Kick-Tight) :** Garantit la faisabilité après des perturbations. Si une tournée est infaisable, on éjecte itérativement le client ayant la fenêtre de temps la plus étroite jusqu'à restaurer la validité. Les clients éjectés sont réinsérés dans de nouvelles tournées. *Comp:* $\mathcal{O}(m^2)$

— **BestInsertion (Regret-2) with Slack Penalty :** Pour réinsérer un client, on teste p positions dans r tournées avec k véhicules. On priorise les clients ayant le plus grand "regret" (écart de coût entre la meilleure et la seconde meilleure option). *Note :* On pénalise les insertions qui réduisent trop le slack temporel pour éviter de créer des tournées fragiles. *Comp:* $\mathcal{O}(r \cdot k \cdot p \cdot m^2)$

3.4 Opérateurs de Recherche Locale

— **Intra-Route 2-opt :** Inverse une sous-séquence dans une tournée pour réduire la distance. Complexité dominée par le calcul du diamètre. *Comp:* $\mathcal{O}(m^2)$

— **Inter-Route Relocate :** Déplace un client d'une tournée vers la meilleure position possible dans une autre. Équilibre les charges et prépare l'élimination de tournées. *Comp:* $\mathcal{O}(r \cdot k \cdot p \cdot m^2)$

— **Cross-Exchange (1-1) :** Échange un client de la tournée A avec un client de la tournée B. *Comp:* $\mathcal{O}(m^2)$

— **Inter-Route 2-opt* :** Coupe deux tournées et échange leurs fins pour permettre des changements structurels majeurs. *Comp:* $\mathcal{O}(m^2)$

— **Route Elimination (Strategic Merge) :** Cible une petite tournée (taille $s \leq 8$) et tente de la vider en redistribuant tous ses clients dans r autres tournées. C'est le levier principal pour réduire le coût fixe de location. *Comp:* $\mathcal{O}(s \cdot r \cdot k \cdot p \cdot m^2)$ (+ feasibility check)

4 Algorithme Global (Pseudocode)

Algorithm 1 Hybrid LNS-SA Solver

Require: Instance I, Time limit $T_{\max}$
Ensure: Best Solution $\mathcal{S}^\star$

```
S ← GreedyInit(I) ; S ← Repair(S) ; S* ← S ; T ← T_0
while time < T_max do
    S' ← S
    // 1. Phase de Ruin
    if rand() < P_random then RuinRandom(S', q)
    else RuinWorstLight(S', q)
    end if
    // 2. Phase de Recréation
    L_rem ← Extract(S') ; RecreateRegret2(S', L_rem)
    Repair(S')
    // 3. Phase de recherche locale
    Apply2Opt(S') ; ApplyRelocate(S')
    ApplyCrossExchange(S') ; Apply2OptStar(S')
    RouteElimination(S')
    Repair(S')
    // 4. Critère d'Acceptation (Recuit Simulé)
    ΔC ← Cost(S') − Cost(S)
    if ΔC < 0 or exp(−ΔC/T) > rand(0,1) then
        S ← S' ;
        if Cost(S) < Cost(S*) then S* ← S
        end if
    end if
    T ← T × α                                   ▷ Cooling
end while
return S*
```

Pour l'algorithme final nous avons pris : $\alpha = 0.99965$ et $P_{random} = 0.78$

5 Stratégie de Borne Inférieure (Lower Bound)

Pour évaluer la performance, nous avons calculé une Borne Inférieure (LB) en relaxant les contraintes les plus dures (Fenêtres de temps et Diamètre). Le problème est décomposé en deux sous-problèmes résolus à l'optimalité grâce au solveur HiGHS :

1. LB Location (LB_{rent}) : Unbounded Knapsack Problem (MILP) cherchant le coût de flotte minimal pour couvrir la demande totale.
2. LB Carburant (LB_{fuel}) : Degree-2 Relaxation (LP). Nous minimisons la somme des arêtes pondérées telle que chaque client ait un degré 2 (en ignorant les sous-tours), pondérée par le coût carburant minimal.

$$\textbf{Total LB} = LB_{rent} + LB_{fuel}$$

6 Resultats numériques

Le Tableau 1 présente nos résultats. Le gap total est une moyenne pondérée des gap par instance.

Table 1 – Best Feasible Cost vs Relaxed Lower Bound

Instance	**Lower Bound**	**Best Cost**	**Gap (%)**	**Status**
01	404.09	444.39	9.97	Feasible
02	693.65	759.42	9.48	Feasible
03	836.04	955.94	14.34	Feasible
04	1264.10	1461.17	15.59	Feasible
05	1556.83	1778.24	14.22	Feasible
06	1978.61	2287.61	15.62	Feasible
07	2527.01	3164.75	25.24	Feasible
08	3756.43	4459.43	18.71	Feasible
09	4586.67	5490.96	19.72	Feasible
10	5818.54	6873.93	18.14	Feasible
Total	**23413.74**	**27675.84**	**18.20%**	-

Conclusion : Le solveur atteint un gap moyen pondéré de **18.20%**. Ce gap est conservateur car la LB ignore les fenêtres de temps (principale difficulté). Le faible gap (<10%) sur les instances 1-2 valide l'efficacité de l'approche.